Python Engineering - Class 05

Listas, tuplas y trayectorias | Guía de ejercicios

Objetivo de la clase: Aprender a almacenar colecciones de estados y entender mutabilidad, referencias y snapshots de una trayectoria.

Este documento es la guía base del ejercicio. No contiene una solución completa para copiar: organiza el problema, los pasos de implementación, los valores de referencia y la validación.

Ruta

```
01_python_bootcamp/
`-- week_02_data_structures/class_05_lists_tuples/
    |-- notes/
    `-- exercices/
```

Flujo de trabajo obligatorio

```
PROBLEM
INPUTS
OUTPUTS
EQUATIONS
SPECIAL CASES
DO I NEED ITERATION?
ALGORITHM
VALIDATION
```

Antes de programar, redacta estas secciones. El objetivo es separar el razonamiento de la sintaxis.

EX13 - Trayectoria con listas y tuplas

Problema: Repetir la simulacion vertical y almacenar cada estado como una tupla (h,v,t) dentro de una lista.

Entradas de referencia: h0=0, v0=50, t0=0, g=9.81, dt=0.1

Ecuaciones / relación: Movimiento vertical Euler + validación analítica

Comprobación esperada: trajectory debe ser una list; cada estado una tuple; las magnitudes extraídas de trajectory deben ser coherentes con las calculadas.

Guía de implementación

- 1. Crear listas de h,v,t y trajectory.
- 2. append del estado inicial.
- 3. Iterar con Euler.
- 4. append de una tuple nueva en cada paso.
- 5. Extraer maximo/final usando la trayectoria.

Antes de ejecutar

Predice el signo, el orden de magnitud, el tipo de dato y cualquier shape relevante. Después compara esa predicción con la salida real.

Validación final

Responde estas preguntas como comentarios al final del ejercicio. El objetivo es demostrar que puedes explicar el flujo del código.

- Que diferencia hay entre list y tuple?
- Que hace append()?
- Que significa trajectory[-1]?
- Por que guardar tuples nuevas evita perder estados anteriores?
- Que es aliasing?
- Que devuelve max(state[0] for state in trajectory)?

Criterio de cierre

La clase se considera completada cuando el código se ejecuta, las comprobaciones de referencia tienen sentido, puedes explicar cada bloque sin leer una solución y puedes modificar al menos un input anticipando su efecto.